ISLAMIC UNIVERSITY OF TECHNOLOGY

SWE 4301

LECTURE 2

Topic: Introduction to TDD

November 24, 2025

PREPARED BY

Maliha Noushin Raida

Lecturer, Department of CSE

Contents

1 Introduction to TDD **3**

1.1 Understanding Testing 3

1.2 What is unit testing? 4

1.3 Writing the First Test 4

1.4 Best Practices for TDD 8

1 INTRODUCTION TO TDD

What is readable code? A code that correctly reflects the problem it is solving. In previous class, we have learnt that writing clean code is like drawing a picture. But there is a question: whether we all can write clean, shamelessly green code. One framework to use for writing green code is called TDD (Test driven Development).

1.1 Understanding Testing

A generation ago, a handful of extreme programming (XP) practitioners began writing automated tests using a technique they called "test first development." Their ideas were so influential that automated tests are now the norm, and these tests are often written first, in prelude to writing code. Example:

```
I expect the table to be four feet in diameter.
The test fails because I have no table.
I cut a circular piece of wood four feet in diameter.
The test passes.
------------------------------

I expect the table to be three feet high.
The test fails because it is sitting on the ground.
I add one leg in the middle of the table.
The test passes.
------------------------------

I expect the table to hold a 20-pound object.
The test fails because when I place the object on the edge,
it makes the table fall over since there is only one leg in the middle.
I move the one leg to the outer edge of the table and
add two more legs to create a tripod structure.
The test passes
------------------------------
```

The practice of writing tests before writing code became known as test-driven development (TDD). In its simplest form, TDD works like this:

- **Write a test.** Because the code does not yet exist, this test fails. Test runners usually display failing tests in red.
- **Make it run**. Write the code to make the test pass. Test runners commonly display passing tests in green.
- **Make it right**. Each time you return to green, you can refactor any code into a better shape, confident that it remains correct if the tests continue to pass.

In this cycle, you must repeatedly decide which test to write next, how to arrange code so that the test passes, and how much refactoring to do once it does. Each decision requires judgment and has consequences.

1.2 What is unit testing?

Unit tests are automated and are run each time the code is changed to ensure that new code does not break existing functionality. Unit tests are designed to validate the smallest possible unit of code, such as a function or a method, and test it in isolation from the rest of the system. This allows developers to quickly identify and fix any issues early in the development process, improving the overall quality of the software and reducing the time required for later testing.

1.3 Writing the First Test

The first test is often the most difficult to write. At this point, you know the least about whatever it is that you intend to do. Your problem is a big, fuzzy, amorphous blob, and it's challenging to reach in and carve off a single piece. It feels important to choose well because where you start informs how you'll proceed and ultimately determines where you'll end.

Despite its apparent import, the choice you make here hardly matters. In the beginning, you have ideas about the problem but actually know very little. You can't figure out what's right until you write some tests, at which time you may realize that the best course of action is to throw everything away and start over. So, while it is important to consider the problem and to sketch out an overall plan before writing the first test, don't overthink it. Find a starting place and get going, in faith that as you proceed, the fog will clear.

Testing is the process of ensuring a program receives the correct input and generates the correct output and intended side-effects.

This test, like all tests, contains three parts:

- Setup Create the specific environment required for the test.
- Do Perform the action to be tested.
- Verify Confirm the result is as expected.

TDD tells you to write the simplest code that will pass this test. In this case, your goal is to write only enough code to change the error message

When doing TDD, you toggle between wearing two hats. While in the "writing tests" hat, you keep your eye on the big picture and work your way forward with the overall plan in mind. When in the "writing code" hat, you pretend to know nothing other than the requirements specified by the tests at hand. Thus, although each individual test is correct, until all are written, the code is incomplete.

Example:

Requirements:

- For a given string "AB" should return "BA".
- For a given string "ABCD" should return "ABDC".
- For a given string "SELENIUM" should return "SELENIMU".
- For a given string "A" should return "A". (If there is only a single char in a string it should return the same char.)
- Empty string should return empty.

1. First test for the example,

```
public class StringSwap {
   @Test
   public void stringSwap2charsOnly() {
         StringSwapHelper help=new StringSwapHelper ();
         Assert.assertEquals("BA", help.swaplasttwochars("AB"));
   }
}
```

we have created an instance of class StringSwapHelper which does not exist. If this test is run, it will give compilation error and fail the test case. (**RED**)

2. Let us go ahead and create the StringSwapHelper class inside a package under src/main/-java

```java
public class StringSwapHelper {
   public String swaplasttwochars(String string) {

         // TODO Auto-generated method stub
         return null;
   }
}
```

If we run the test case, it will still fail as the swaplasttwocharsclass() method is returning null. 3. Now we should add the business logic and what we expect the swaplasttwocharsclass() method to perform. We will add the logic to swap for only 2 characters first.

```java
public class StringSwapHelper {
   public String swaplasttwochars(String string) {
         char firstChar = string.charAt(0);
         char secChar = string.charAt(1);
         return "" + secChar + firstChar; // "" is appended as 2 chars
   cannot be appended
   }
}
```

Run the test and observe that the test case is passed. (**GREEN**) 4. Now, we need to write more test cases to test out other conditions,

```java
    @Test
   public void stringSwap4chars () {
         StringSwapHelper help=new StringSwapHelper();
         Assert.assertEquals("ABDC", help.swaplasttwochars("ABCD"));
   }
```

Run the test, it will fail, right? 5. Then Refactor our code,

```java
    public String swaplasttwochars(String string) {
         int length = string.length();
         String strExceptLast2Chars = string.substring(0, length - 2);
         char secondLastChar = string.charAt(length - 2);
         char lastChar = string.charAt(length - 1);
         return strExceptLast2Chars + lastChar + secondLastChar;
   }
```

6. For a full string,

```java
    @Test
   public void stringSwap() {
         StringSwapHelper help=new StringSwapHelper();
```

```
        Assert.assertEquals("SELENIMU",help.swaplasttwochars("SELENIUM"));
    }
```

7. For one character,

```
    @Test
    public void stringSwap1CharOnly() {
        StringSwapHelper help=new StringSwapHelper();
        Assert.assertEquals("A", help.swaplasttwochars("A"));
    }
```

This test returns an exception of StringIndexOutOfBoundsException. 8. Again refactor for single character,

```
    public String swaplasttwochars(String string) {
        int length = string.length();
        if(length==1) {
            return string;
        }
        String strExceptLast2Chars = string.substring(0, length - 2);
        char secondLastChar = string.charAt(length - 2);
        char lastChar = string.charAt(length - 1);
        return strExceptLast2Chars + lastChar + secondLastChar;
    }
```

9. For empty string,

```
    @Test
    public void stringSwap1CharOnly() {
        StringSwapHelper help=new StringSwapHelper();
        Assert.assertEquals("", help.swaplasttwochars(""));
    }
```

Again, it will return an exception of StringIndexOutOfBoundsException. 10. Refactor it again,

```
    public String swaplasttwochars(String string) {
        int length = string.length();
        if(length<=1) {
            return string;
        }
        String strExceptLast2Chars = string.substring(0, length - 2);
        char secondLastChar = string.charAt(length - 2);
        char lastChar = string.charAt(length - 1);
        return strExceptLast2Chars + lastChar + secondLastChar;
    }
```

Kent Beck describes different ways to make tests pass. Three of his "Green Bar Patterns" are:

- Fake It ("Til You Make It") - Return a constant and gradually replace constants with variables until you have the real code. Write enough code to pass the test.
- Obvious Implementation - Type in the real implementation. The small steps of TDD act to incrementally reveal the correct implementation. If your absolute certainty turns out to be wrong, skipping these incremental steps means you miss the opportunity of being set right.
- Triangulate - described as a way to “conservatively drive abstraction with tests.” Triangulation requires writing several tests at once, which means you’ll have multiple simultaneous broken tests. The idea is to write one bit of code which makes all of the tests pass. Triangulation is meant to force you to converge upon the correct abstraction in your code. You can expose a common, underlying abstraction through the accumulation of multiple concrete examples.

1.4 Best Practices for TDD

- **Write small test cases which fail at the start:** Start by writing small test cases that capture the actual behavior of the functionality. Write a test case such that it fails the expected behavior confirming that the test is actually testing the functionality.
- **Tests should be independent:** Focus on one functionality at a time and progress in incremental manner. Every test should be self-sufficient and capable of working in isolation. It should not be dependent on any other test case to avoid complexity.
- **Execute tests regularly:** A test should be conducted before coding and after coding. It should also be performed after code refactoring. In simple words, run the test for any code change or code update to ensure that any regressions if found should be fixed on time and the development pace should be on track.
- **Tests for positive and negative scenarios:** Tests for positive scenarios are fine however, tests for negative scenarios are also important. This surfaces any potential issues with the code and makes the system robust.

TDD is a discipline for programmers like double-entry bookkeeping is for accountants or sterile procedure is for surgeons.

- Robert C. Martin